

Simon Zhu

Project Portfolio | Mechatronics Engineering, University of Waterloo

Looking for a Winter 2027 co-op

This PDF is the short version. The full portfolio is a website.

It has the real CAD of every build in 3D that you can rotate, videos of the machines running, the chess engine running in your browser so you can play it, and the trained robot controller replayed in simulation.

→ simonnnnz.github.io

CONTENTS

- 01 **Check-Mate, the automated chess board** Hall-effect sensor grid, CoreXY gantry, custom H-bridge, onboard engine
- 02 **CRC competition robot** Modular swerve drive and dual claw, 2nd place for Robot Design
- 03 **Wheel-leg biped** Reinforcement-learning balance controller in MuJoCo, hardware in progress

01 DEC 2023 TO MAR 2025

Check-Mate, the Automated Chess Board

Interactive version: [rotate all three builds, drag the gantry, play the engine](#)

A real chess board that plays against you by itself. A Hall-effect sensor under every square reads where the pieces are, and a CoreXY gantry carries an electromagnet under the board to move the engine's pieces. H.G. Muller's micro-Max chess engine, which I adapted for the machine, runs directly on the Arduino with no computer in the loop. I designed it for seniors, so every move is also announced out loud.

64

Hall-effect sensors

250+

individual components

40 m

of wiring

3

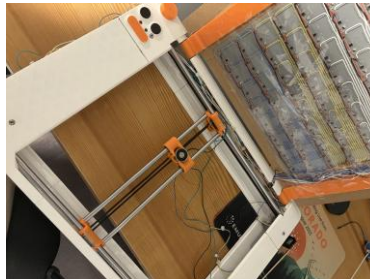
full builds in 18 months

2 KB

the entire chess engine



MK3, render



MK3 open, electronics under the lid



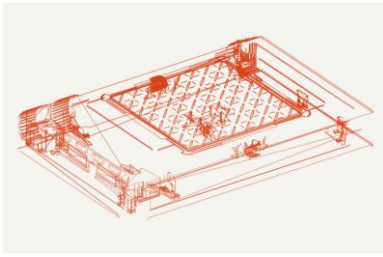
Hydro-Québec Science Fair

Provincial finalist, Hydro-Québec Science Fair. OIQ (Ordre des ingénieurs du Québec) award for technical complexity, OCTAS Youth Innovation award, and a \$2,000 Université de Sherbrooke scholarship.

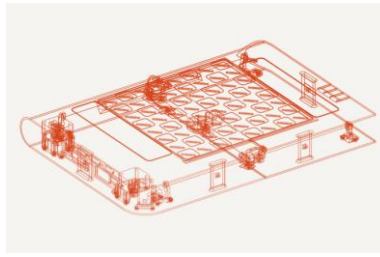
How it works

- **Sensing.** Each square has a linear Hall-effect sensor, which reads the magnet's polarity as well as its presence, so the board knows the colour of every piece instead of tracking pieces from memory. The 64 signals go through four 16-channel analog multiplexers that share their select lines.
- **Movement.** A CoreXY gantry keeps both motors fixed to the frame, so neither motor drives X or Y on its own and every move is a combination of the two. The move planner turns a chess move into motor steps, with routes that slide a piece between the other pieces without knocking them, and moving captured pieces off the board.
- **Engine.** I modified micro-Max to take moves from the sensor grid, check that they are legal, and adjust difficulty through search depth, evaluation weights and randomness.
- **Features.** An LED grid under the board lights the legal moves for a piece you pick up and the moves the engine would pick, the speaker announces every move, and the board resets and calibrates itself.

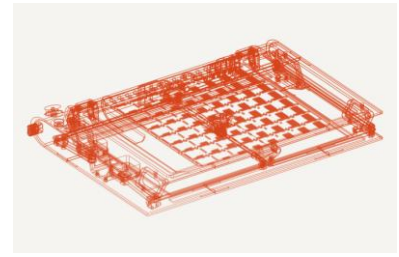
Three builds to get here



MK1, concept



MK2, refined CoreXY



MK3, Hall sensors

- **MK1** was a concept build to work out the base dimensions, find the main mechanical problems and settle a design language.
- **MK2** fit all of the electronics, improved the CoreXY system, added housing and support for the playing surface, and was designed to be manufactured and assembled.
- **MK3** replaced the reed switches with Hall-effect sensors to stop false readings from floating switches, grew the play area toward a full-size set, and added cooling, controls, LEDs, a speaker and a playing surface that lifts for maintenance.

What I had to work out

- **Electromagnetic interference.** The motors, drivers and the electromagnet all threw noise across the sensor grid. Every motor and sensor signal wire is shielded, and each of the 64 sensors is calibrated both with no interference and with magnets present.
- **Driving the electromagnet.** Commercial motor drivers kept failing because of the back EMF from the electromagnet. I designed and built a 4-MOSFET H-bridge with a large diode for the inductive kickback. The MOSFETs were not turning fully on from 5 V logic, so I added two boost converters to drive the gates at 8 V.
- **Heat.** The frame is 3D-printed PLA, which softens around 65 °C. A thermistor watches the temperature inside and turns on cooling before the parts get close to that.
- **Packaging.** Fitting the gantry, 64 sensors, the multiplexers and 40 m of wire into a board under 2 kg meant mounting components on every surface, including under the hinged playing surface.
- **The night before judging.** A wiring fault set the board on fire. I isolated the fault and repaired it overnight.

02 SEP 2025 TO MAR 2026

CRC Competition Robot

Interactive version: [take the swerve module apart in 3D](#), [see the build gallery](#)

Over eight months I developed the robot for John Abbott College's team at the Canadian Robotics Competition, as club president and robot design lead. I built a swerve drive drivetrain and a dual claw that stacks and moves game pieces, and can move in all three axes independently of the drive base. The robot placed 2nd for Robot Design.

2nd

Robot Design, CRC

\$50

per module, vs ~\$400
bought

30 s

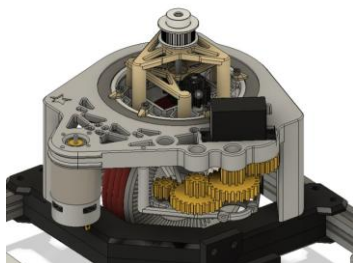
to swap a module

4

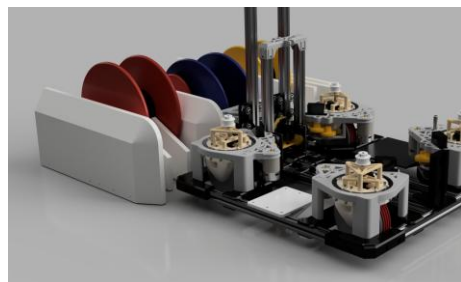
bolts, the only tools
needed

5

design iterations



Swerve module, CAD



Whole robot, render



Drive base during the build

Swerve drive

A swerve drive is a powered wheel that can also steer a full 360 degrees. Syncing the steering and drive of all four wheels lets the robot drive in any direction without changing where it faces.

- **Designed to be swapped, not repaired.** Each module is self-contained and held in by four bolts. Take them out and the whole module drops out from below, and a fresh one goes in the same way.
- **Coaxial and fully 3D-printed.** The drive passes through the steering axis, so the wheel can keep spinning while the module turns. Commercial modules cost about \$400 each, and mine cost about \$50.
- **Closed-loop steering.** Optical encoders on continuous-rotation servos close the steering loop. Every module's angle has to be zeroed against a known reference, and the code converts one joystick command into a steering angle and a wheel speed for each of the four modules, so they stay in sync while driving.
- **Five versions.** Each one was printed, assembled, driven, then taken apart and redesigned before the next.

Claw: two chopsticks, one game piece

The claw does not grip. Two arms on fixed pivots act like chopsticks, each staying tangent to the game piece, so swinging them pushes the piece side to side while it is held from both directions the whole time. The game scores a multiplier for stacked pieces, so placement has to be exact, and the chopstick motion lines a piece up over the stack while the base stays still.

Running the team

- Led a 30+ member club with a \$6,000 budget, and managed timelines, submission deadlines and task allocation.
- Taught weekly workshops in CAD (Fusion 360), C++ and manufacturing.

03 AUG 2026 TO PRESENT

Wheel-Leg Biped, Balance Controller

Interactive version: [watch the trained policy drive, see the network fire live](#)

A small wheel-leg balancing robot, about 1 kg, with six motors: two direct-drive wheels, and two motors per leg driving a 5-bar linkage so body pitch can be controlled separately from wheel torque. Instead of tuning a controller by hand, I trained a neural network with reinforcement learning (PPO) in the MuJoCo physics simulator. The hardware is being built now.

22,406

weights in the network

25 M

control steps of
practice

10.8 h

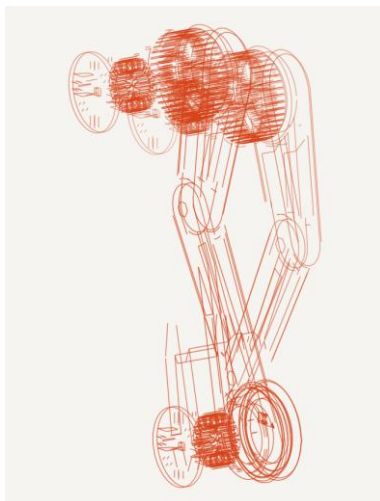
of training on a laptop
CPU

100 Hz

control rate

39 / 6

network inputs / outputs



Leg, from the SolidWorks CAD

Hardware

- **Motors and drivers.** Six 2208 gimbal motors with field-oriented control on SimpleFOC Mini boards, with magnetic encoders on every motor.
- **Why gimbal motors.** The drivers have no current sensing and about 0.8 Ω of on-resistance, so a low-resistance motor would make torque control meaningless and dump the heat into the driver. High-resistance windings were a hard requirement.
- **Motor measurements.** I measured the phase resistance of all six motors on the bench (9.2 to 9.55 Ω) and the torque constant, which came out 8.5 times lower than the figure I first calculated. The simulation uses the measured values.
- **Control.** Split across two ESP32-S3s because of pin count, with an IMU and a Kalman filter for body angle, and 3D-printed planetary gearboxes on the legs. The battery is two 2S packs in series.

Training

- Every 10 ms the network takes 39 values (gravity direction, gyro, hip angles and speeds, wheel speeds, motor temperatures, the command) and returns 6 motor commands. Hip targets go to a 1 kHz PD loop, and wheels are commanded in torque.

- Six simulators run in parallel. The reward pays for tracking speed, turn rate and ride height and staying upright, and penalizes jerky commands, overheating the motors, sliding and falling.
- A curriculum starts on flat ground with an ideal robot, then over the first 40% of training adds rougher terrain, harder pushes and wider randomization of mass, friction, gearbox efficiency, backlash, latency, sensor error and supply voltage.
- The weights export to a C header for the microcontroller. I checked the C forward pass against the Python network, and they match to within 4×10^{-7} .

What I had to work out

- **Checking why episodes ended.** Episode length dropped from 1860 to 1160 steps partway through a run. It looked like the normal dip when the curriculum gets harder, but the crouch limit had been calculated from nominal motor values, and 92.5% of the randomized robots physically could not hold the commanded pose.
- **A frame bug.** The velocity reward was computed in the world frame instead of the robot's, so it measured the wrong axis whenever the robot turned. Forward speed tracking was stuck at 52% until I fixed it.
- **Exporting the weights.** The network only works with the input statistics it was trained with. An export without them passed its own check but made the robot behave wrongly, so the export script now refuses to run without them.

Current status. In simulation it balances, drives, turns in place and recovers from pushes. Forward speed still reaches only about two thirds of a fast command and it does not follow ride height yet. Next is more validation in simulation, then the real robot.

TOOLS I USED ACROSS THESE PROJECTS

CAD and manufacturing: SolidWorks, Fusion 360 (7 years), AutoCAD, 3D printing, soldering, rapid prototyping

Electronics and embedded: Arduino, ESP32-S3, SimpleFOC, SPI, UART, I2C, multiplexers, MOSFET H-bridges, encoders, IMUs

Software and control: C++, Python, MuJoCo, PyTorch and Stable-Baselines3 (PPO), PID, field-oriented control, Kalman filter

See all of this running.

Rotate the CAD of every build, watch the swerve module come apart, play the chess engine, and watch the trained biped controller drive in your browser. Also in progress on the site: the biped hardware and a flying probe tester.

→ simonnnz.github.io